

Rapport TP - Environnement Wazuh sous Docker

Mathyves Meranville

20 mars 2026

Introduction

Dans ce rapport, je décris la démarche que j'ai suivie pour mettre en place un environnement Wazuh entièrement sous Docker, avec un manager et deux agents basés sur des distributions Linux différentes.

Dépôt Git : <https://github.com/Thyvesma/wazuh-tp.git> (branche `main`)

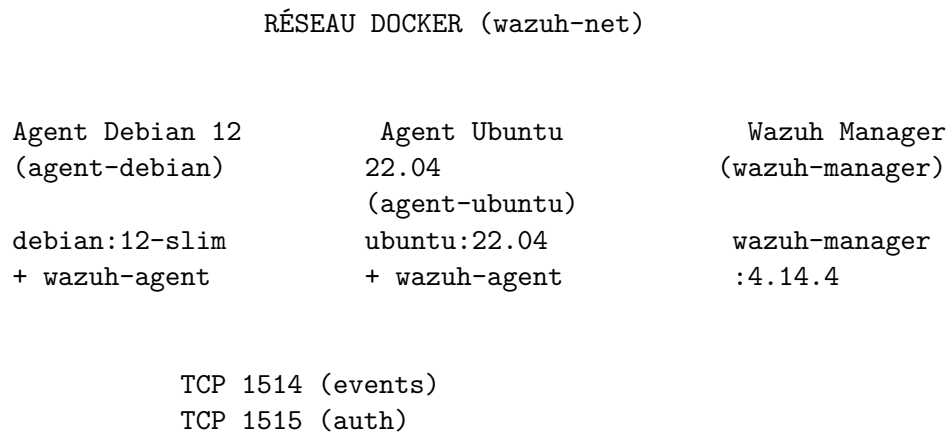
Sommaire

1. Description de l'architecture
 - 1.1 Schéma d'architecture
 - 1.2 Liste des images Docker et versions
2. Fichiers de déploiement
 - 2.1 Structure du projet
 - 2.2 Ports exposés
 - 2.3 Volumes (persistance)
 - 2.4 Variables d'environnement principales
3. Mise en œuvre pratique
 - 3.1 Prérequis
 - 3.2 Étapes de déploiement
 - 3.3 Commandes de contrôle
4. Validation fonctionnelle
 - 4.1 Vérification de la remontée des événements
 - 4.2 Génération d'événements de test
 - 4.3 Format des alertes (alerts.json)
5. Analyse des remontées Wazuh
 - 5.1 Chaîne de traitement
 - 5.2 Types d'événements observés
 - 5.3 Exemples de cas concrets détectés
6. Sécurité et isolation
 - 6.1 Isolation des conteneurs
 - 6.2 Contrainte respectée
 - 6.3 Limites sans indexer/dashboard
7. Annexes
 - 7.1 Dépôt Git
 - 7.2 Commandes Docker utiles
 - 7.3 Extrait docker-compose.yml
 - 7.4 Extrait wazuh_manager.conf

1. Description de l'architecture

J'ai conçu une architecture simple où deux agents (Debian et Ubuntu) envoient leurs événements vers un manager unique via un réseau Docker partagé.

1.1 Schéma d'architecture



Ports exposés : 1514, 1515, 514, 55000

1.2 Liste des images Docker et versions

Composant	Image de base	Version Wazuh	Rôle
Wazuh Manager	wazuh/wazuh-manager	4.14.4	Réception des événements, analyse, règles
Agent Debian	debian :12-slim	Agent 4.14.4-1	Collecte événements (Debian)
Agent Ubuntu	ubuntu :22.04	Agent 4.14.4-1	Collecte événements (Ubuntu)

Note : Conformément au sujet, je n'ai déployé aucun indexer (Elasticsearch/OpenSearch) ni dashboard (Kibana). La validation s'effectue via les fichiers de logs et d'alertes du manager.

↑ Retour au sommaire

2. Fichiers de déploiement

J'ai créé les fichiers suivants pour le déploiement. La structure complète est disponible dans le dépôt Git.

2.1 Structure du projet

```
wazuh/
  docker-compose.yml      # Orchestration des services
  config/
    wazuh_manager.conf    # Configuration manager (indexer désactivé)
    certs/                # Certificats SSL (générés par script)
      root-ca.pem
      filebeat.pem
      filebeat.key
  agents/
    agent-debian/
      Dockerfile
      entrypoint.sh
    agent-ubuntu/
      Dockerfile
      entrypoint.sh
  scripts/
    generate-certs.sh      # Génération des certificats
    enroll-agents.sh       # Aide à l'enrôlement
    test-events.sh         # Génération d'événements de test
  RAPPORT_TP_WAZUH.md     # Ce rapport
```

2.2 Ports exposés

Port	Service	Usage
1514	Wazuh Manager	Connexion des agents (TCP, événements)
1515	Wazuh Manager	Enrôlement des agents (auth)
514	Wazuh Manager	Réception Syslog (UDP)
55000	Wazuh Manager	API Wazuh

2.3 Volumes (persistance)

- **wazuh_logs** : Logs et alertes du manager (/var/ossec/logs)
- **wazuh_etc** : Configuration persistante
- **wazuh_queue** : File d'attente des événements
- Autres volumes pour intégrations, wodles, etc.

2.4 Variables d'environnement principales

- **Manager** : INDEXER_URL, API_USERNAME, API_PASSWORD, chemins certificats
- **Agents** : WAZUH_MANAGER=wazuh-manager (hostname du service)

↑ Retour au sommaire

3. Mise en œuvre pratique

3.1 Prérequis

J'ai utilisé Docker Engine et Docker Compose, avec un accès réseau pour télécharger les images.

3.2 Étapes de déploiement

Étape 1 : Générer les certificats

```
cd /chemin/vers/wazuh
chmod +x scripts/generate-certs.sh
./scripts/generate-certs.sh
```

Étape 2 : Construire et lancer les services

```
# Construction des images des agents
docker compose build
```

```
# Démarrage en arrière-plan
docker compose up -d
```

Étape 3 : Vérifier le démarrage

```
# État des conteneurs
docker compose ps
```

```
# Logs du manager (attendre ~2 min pour le démarrage complet)
docker compose logs -f wazuh-manager
```

Étape 4 : Enrôlement des agents

Méthode recommandée (agent-auth) :

```
# Depuis l'agent Debian
docker exec -it wazuh-agent-debian /var/ossec/bin/agent-auth -m wazuh-manager
```

```
# Depuis l'agent Ubuntu
docker exec -it wazuh-agent-ubuntu /var/ossec/bin/agent-auth -m wazuh-manager
```

Méthode alternative (manage_agents sur le manager) :

```
# Lister les agents
docker exec -it wazuh-manager /var/ossec/bin/manage_agents -l
```

```
# Ajouter un agent (interactif)
docker exec -it wazuh-manager /var/ossec/bin/manage_agents -a
```

```
# Importer la clé dans l'agent
docker exec -it wazuh-agent-debian /var/ossec/bin/manage_agents -i <CLE_GENEREE>
```

```
# Valider l'agent
docker exec -it wazuh-manager /var/ossec/bin/manage_agents -e <ID>
```

Étape 5 : Accéder aux conteneurs

Shell dans le manager

```
docker exec -it wazuh-manager /bin/bash
```

Shell dans un agent

```
docker exec -it wazuh-agent-debian /bin/bash
```

```
docker exec -it wazuh-agent-ubuntu /bin/bash
```

3.3 Commandes de contrôle

Statut des services Wazuh dans le manager

```
docker exec -it wazuh-manager /var/ossec/bin/ossec-control status
```

Liste des agents connectés

```
docker exec -it wazuh-manager /var/ossec/bin/agent_control -l
```

↑ Retour au sommaire

4. Validation fonctionnelle

4.1 Vérification de la remontée des événements

Fichiers à consulter sur le manager :

Alertes JSON (temps réel)

```
docker exec -it wazuh-manager tail -F /var/ossec/logs/alerts/alerts.json
```

Alertes texte

```
docker exec -it wazuh-manager tail -F /var/ossec/logs/alerts/alerts.log
```

Logs du manager

```
docker exec -it wazuh-manager tail -F /var/ossec/logs/ossec.log
```

4.2 Génération d'événements de test

Exécuter dans chaque conteneur agent :

Agent Debian - Création de fichier

```
docker exec -it wazuh-agent-debian touch /tmp/wazuh-test-$(date +%s).txt
```

Agent Ubuntu - Création de fichier

```
docker exec -it wazuh-agent-ubuntu touch /tmp/wazuh-test-$(date +%s).txt
```

Modification dans /etc (zone surveillée par syscheck)

```
docker exec -it wazuh-agent-debian bash -c "echo test >> /tmp/wazuh-test-modification.txt"
```

4.3 Format des alertes (alerts.json)

Chaque ligne est un objet JSON contenant notamment : - `timestamp` : Date/heure - `rule.id` : Identifiant de la règle - `rule.description` : Description - `rule.level` : Niveau de criticité - `agent.name` : Nom de l'agent source - `full_log` : Log complet

↑ Retour au sommaire

5. Analyse des remontées Wazuh

5.1 Chaîne de traitement

Agent (collecte) → Envoi TCP 1514 → Manager (analyse) → Moteur de règles
↓
alerts.json / alerts.log

5.2 Types d'événements observés

Type	Règle typique	Niveau	Description
Syscheck - Nouveau fichier	550	7	Fichier créé dans zone surveillée
Syscheck - Modification	554	7	Fichier modifié
Connexion agent	501	3	Agent connecté au manager
Déconnexion agent	502	3	Agent déconnecté
Rootcheck	510-516	7-12	Détection rootkit

5.3 Exemples de cas concrets détectés

1. **Règle 550 - Syscheck new file** : Création d'un fichier dans `/tmp` ou `/etc` déclenche une alerte car ces répertoires sont surveillés par l'intégrité des fichiers.
2. **Règle 554 - Syscheck file modified** : Modification d'un fichier existant dans une zone surveillée (checksum modifié).
3. **Règle 501 - Agent connected** : Chaque fois qu'un agent s'enregistre ou se reconnecte au manager.

[↑ Retour au sommaire](#)

6. Sécurité et isolation

6.1 Isolation des conteneurs

- Chaque service tourne dans son propre conteneur
- Réseau Docker dédié (`wazuh-net`)
- Aucune modification de l'hôte : tout est confiné dans Docker
- Les volumes persistent les données sans exposer le système hôte

6.2 Contrainte respectée

J'ai respecté strictement la contrainte : aucune action de durcissement, logrotate ou configuration Wazuh n'est appliquée sur l'hôte. Toutes les manipulations sont effectuées uniquement dans les conteneurs.

6.3 Limites sans indexer/dashboard

- **Visibilité réduite** : Pas d'interface graphique pour visualiser les alertes
- **Analyse historique limitée** : Recherche manuelle dans les fichiers JSON
- **Agrégation/filtrage** : Outils en ligne de commande (grep, jq) nécessaires
- **Scalabilité** : Non adapté à un grand nombre d'agents

[↑ Retour au sommaire](#)

7. Annexes

7.1 Dépôt Git

J'ai versionné ce projet sur GitHub. Pour le cloner et reproduire l'environnement :

```
git clone https://github.com/Thyvesma/wazuh-tp.git
cd wazuh-tp
./scripts/generate-certs.sh
docker compose up -d
```

7.2 Commandes Docker utiles

Démarrer

```
docker compose up -d
```

Arrêter

```
docker compose down
```

Logs

```
docker compose logs -f wazuh-manager
docker compose logs -f wazuh-agent-debian
```

Reconstruire

```
docker compose build --no-cache
docker compose up -d --build
```

7.3 Extrait docker-compose.yml

Voir le fichier `docker-compose.yml` à la racine du projet.

7.4 Extrait wazuh_manager.conf (indexer désactivé)

```
<indexer>
  <enabled>no</enabled>
</indexer>
```

7.5 Dockerfiles des agents

Voir `agents/agent-debian/Dockerfile` et `agents/agent-ubuntu/Dockerfile`.

[↑ Retour au sommaire](#)

Conclusion

J'ai mis en place un environnement Wazuh entièrement conteneurisé répondant aux exigences du TP : un manager et deux agents sur des distributions Linux différentes (Debian 12 et Ubuntu 22.04). La remontée des événements vers le manager a été validée via les fichiers `alerts.json` et `alerts.log`. Toutes les configurations et tests restent confinés aux conteneurs Docker, sans modification de l'hôte.

Rapport rédigé le 20 mars 2026 — Mathyves Meranville — TP Wazuh Docker